

Your AI SDD Specification Isn't Incomplete — It Just Lacks DDD — Spec-Driven Development

Five DDD artifacts as spec elements in Spec-Driven Development Level 1: Ubiquitous Language, Bounded Context Canvas, Context Map, Aggregate Spec, ACL — with ready-to-use templates

16 min read · 3 days ago



Jarosław Wasowski

Follow



Listen



Share

... More

“The limits of my language are the limits of my world.” — Ludwig Wittgenstein, Philosopher (*paraphrase of proposition 5.62 from Tractatus Logico-Philosophicus*)

April 2026. Cursor with Claude Opus 4.6 deleted the PocketOS production database and all volume-level backups in 9 seconds — a single API call to Railway. Reading the post-mortem on X on my way to the office, I felt that my own Spec-Driven Development setup was dangerously close to the same failure. We were all missing the same thing — **five files in the /specs folder, without which AI improvises invariants, invents class names, and conflates models across modules.**

This isn't an isolated incident. It's an industry pattern. After six months of writing specifications for Claude Code agents in production, I see that a spec in SDD Level 1 (Spec First) requires **five mandatory documents** if you want AI to deterministically translate it into code. All five come from Eric Evans' 2003 book — Domain-Driven Design (DDD).

This article is a practical guide. I'll show five Markdown templates, filled examples, and the /specs folder structure I've deployed across three SDD projects. No theorizing — files ready to copy.

The /specs Folder Architecture — Seven Files, Two Sources

Spec-Driven Development Level 1 (Spec First) is the lightest SDD variant: you write the specification **before** implementation so AI has a clear goal rather than

guessing. Code remains the primary artifact (the spec may drift after implementation — an acceptable trade-off in L1, unlike Spec-Anchored and Spec-as-Source). The first L1 question is: **what exactly goes in /specs so AI doesn't improvise.**

The answer is five Domain-Driven Design artifacts as spec elements, plus two classic SDD files:

- **constitution.md** — inviolable project principles (architectural DNA from GitHub Spec Kit)
- **domain-terms.md** per Bounded Context — Ubiquitous Language as an authoritative glossary
- **bc-{name}.md** per Bounded Context — Bounded Context Canvas (model boundary, purpose, integrations)
- **context-map.md** — Context Map with relationships between all BCs
- **aggregate-{name}.md** per Core Aggregate — Aggregate Design Spec with explicit invariants
- **acl-{external-system}.md** per external integration — Anti-Corruption Layer Spec
- **requirements.md** per feature — User Stories with Acceptance Criteria in EARS notation

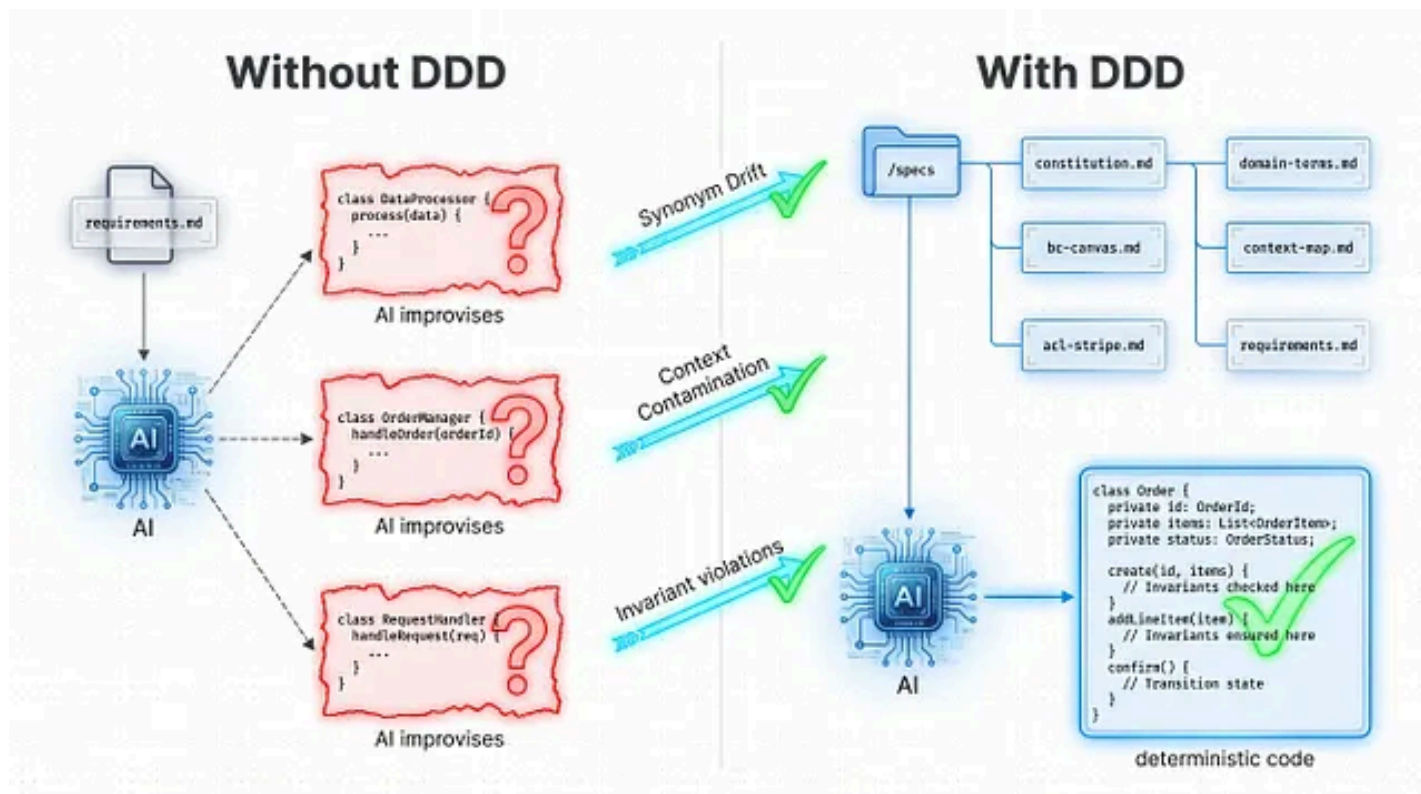
The first five are DDD artifacts. The remaining two

— **constitution.md** and **requirements.md** — are classic SDD files, orthogonal to DDD: the first defines project meta-principles (layered architecture, conventions, language constraints), the second describes a specific feature in User Stories and Acceptance Criteria. DDD doesn't replace them — it complements them with a semantic layer. Without that layer, **requirements.md** with EARS is ambiguous because "Order" in one requirement is an invoice, in another a picking list, in a third a purchase intent. **DDD provides the semantic layer that EARS alone cannot.**

In the sections that follow, we'll walk through each of the five DDD artifacts — with a ready-to-use markdown template and a filled example for the e-commerce domain.

Why DDD and Not Other Patterns — A Brief Diagnosis

A brief diagnosis before we get to the templates. **AI is an amplifier, not a developer.** Give it an ambiguous spec and it amplifies chaos. Give it a precise one and it amplifies quality. This is a problem with the abstraction layer at which you write the spec, not a problem with the model.



A Spec First specification without a DDD layer is like a house without foundations — the five artifacts provide the missing semantics.

DDD has one advantage over other modeling methodologies: **it eliminates three specific classes of LLM hallucinations.** Ubiquitous Language eliminates Synonym Drift (the LLM invents `WebhookEvent` / `WebhookHook` / `HookedWebhook` across successive runs). Bounded Context eliminates Context Contamination (the agent conflates "Order" models from three modules into one contradictory response). Aggregate Invariants eliminate business rule violations (`order.TotalAmount += price` without status validation).

Concrete evidence: an LLM-driven DDD case study conducted with FTAPI Software GmbH across three SOTA models (Claude Opus 4.1, Gemini 2.5 Pro, GPT 5.0; arXiv 2603.26244, March 2026) showed that Ubiquitous Language extraction from requirements documents works with all tested models. A structured aggregate spec based on the Codely Aggregate Design Blueprint (reported by the Codely team in

2026 on real projects) raises the quality of AI-generated code from **roughly sixty percent to eighty percent** of the human-written level.

Sean Grove of OpenAI, in his talk “The New Code” (AI Engineer World’s Fair, June 2025), proposed a thesis that can be paraphrased as: **the specification becomes the fundamental unit of programming, and code is its lossy projection**. A specification without the DDD semantic layer is, however, like source code without types — it compiles but protects against nothing that matters.

Artifact 1: Ubiquitous Language → domain-terms.md

Ubiquitous Language (UL) is a shared, rigorous vocabulary where every concept has exactly one meaning within a given boundary. **UL is not documentation — it’s a deterministic contract.**

The file format I use across all SDD projects has four fields plus one critical one. The “Aliases to AVOID” field is the one most teams skip — and without it, the glossary is descriptive, not contractual.

Markdown template:

```
# Domain Terms — Order Management Bounded Context

> Ubiquitous Language for the Order Management BC. Every term below has
> exactly one meaning within this BC. Cross-BC terms are mapped in
> context-map.md.

## Term: Order
**Definition:** Customer's commitment to purchase one or more products
at agreed prices and a specific delivery address.
**Business Context:** Created when the customer confirms the cart.
States: Draft → Confirmed → Fulfilled → Closed | Cancelled.
**Related Terms:** OrderItem, Customer, Address, Money
**Aliases to AVOID:** Purchase, Transaction, Cart, Sale, OrderDraft

## Term: OrderItem
**Definition:** Single line in an Order — a Product with Quantity at a
captured UnitPrice (price snapshot at time of adding).
**Business Context:** Belongs to exactly one Order. Cannot exist
without parent Order.
**Related Terms:** Order, Product, Money, Quantity
**Aliases to AVOID:** LineItem, CartItem, OrderLine, Item

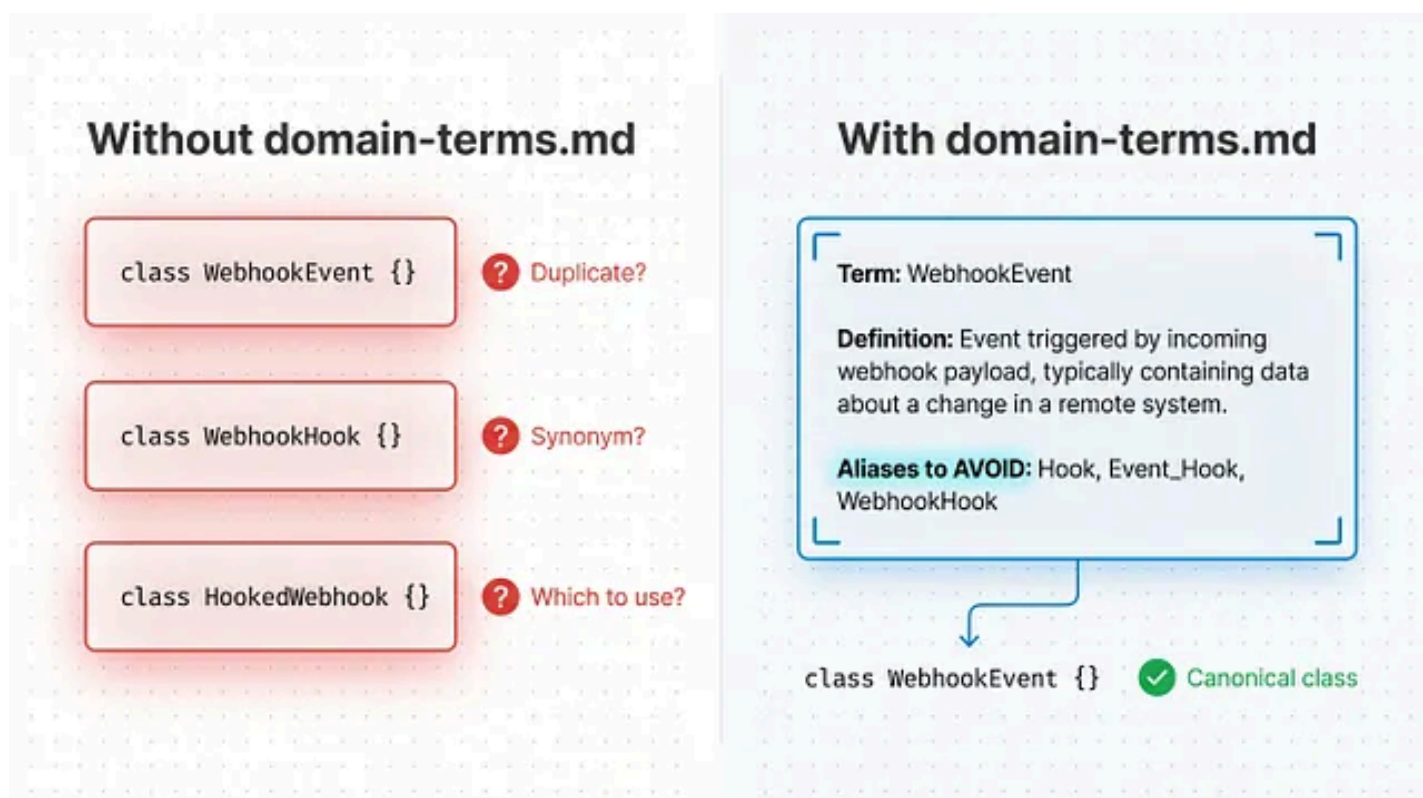
## Term: Confirmed
**Definition:** State of an Order after the customer commits to the
purchase. A Confirmed Order is immutable for items and prices.
```

****Business Context:**** Triggered by ``Order.Confirm()`` command. Emits `OrderConfirmed` domain event.

****Related Terms:**** Order, OrderConfirmed (event)

****Aliases to AVOID:**** Submitted, Placed, Finalized, Validated

A practical test: every word AI invents while writing code **MUST** have a clear 1:1 mapping to a term in `domain-terms.md` or be explicitly banned under "Aliases to AVOID". A class `OrderManager` in the generated code signals a missing glossary entry — that's a flag for refactoring or a code review block.



The "Aliases to AVOID" field turns a descriptive glossary into an executable contract — AI has no right to invent synonyms.

An operational decision: in a new project, the first file is **not a database schema or an API contract** — it's `domain-terms.md` per Bounded Context.

Artifact 2: Bounded Context Canvas → `bc-{name}.md`

A Bounded Context (BC) is a boundary within which the model and language are consistent. The Bounded Context Canvas (Nick Tune, DDD Crew) is a formalized template describing a single BC. It has nine sections — they all matter, but for Spec First L1 I reduce to seven (omitting Verification Metrics and Open Questions, which belong to operations).

Markdown template:

Bounded Context: Order Management

Purpose

Capture customer purchase intent, enforce pricing and stock invariants at order confirmation, emit events for downstream contexts.

Strategic Classification

- ****Domain:**** Core (revenue generator, gives competitive advantage)
- ****Business model:**** Revenue generator
- ****Evolution:**** Custom-built (no off-the-shelf alternative)

Inbound Communication

From context	Channel	Message type	Pattern
Storefront	HTTP API	CreateOrderCommand	Customer-Supplier
Catalog	Event bus	ProductPriceChanged	Conformist
Inventory	HTTP API	StockReserved	Customer-Supplier

Outbound Communication

To context	Channel	Message type	Pattern
Payments	Event bus	OrderConfirmed	Open Host Service
Shipping	Event bus	OrderConfirmed	Open Host Service
Analytics	Event bus	All Order events	Published Language

Ubiquitous Language (summary)

See ``domain-terms.md`` (in this BC folder) for the full glossary.
Key terms: Order, OrderItem, Customer, Money, Quantity, Confirmed.

Business Decisions

- An Order can be Confirmed only once. Re-confirmation is rejected.
- An empty Order (no items) cannot be Confirmed.
- A Confirmed Order is immutable for items, prices, and customer.
- Cancellation is allowed in Confirmed state, prohibited in Fulfilled.

Assumptions

- Pricing is captured at order time (snapshot, not reference to Catalog).
- Inventory is reserved synchronously (no oversell tolerated).
- Customer identity is verified upstream (no auth logic in this BC).

The “Inbound Communication” and “Outbound Communication” sections indicate relationship patterns from the Context Map (Customer-Supplier, Conformist, Open Host Service, ACL, Published Language, Shared Kernel, Separate Ways). They also signal where AI should generate an ACL (when the pattern is Conformist but we want a defensive boundary — in our example, ProductPriceChanged from Catalog).

The “**Business Decisions**” section is the seed of aggregate invariants — we formalize them in `aggregate-order.md`. The “Assumptions” section eliminates AI guesswork: if we capture a price snapshot at order time, the agent won't generate code that fetches the price from the Catalog at runtime.

Artifact 3: Context Map → `context-map.md`

The Context Map documents all BCs in the system and the relationship patterns between them. It's one file per system, not per BC.

Markdown template with a Mermaid diagram and a list of relationship pairs:

```
# Context Map

```mermaid
graph LR
 Storefront[Storefront BC] -->|Customer-Supplier| OM[Order Management BC]
 Catalog[Catalog BC] -.->|Conformist + ACL| OM
 Inventory[Inventory BC] -->|Customer-Supplier| OM
 OM -->|Open Host Service
Published Language| Payments[Payments BC]
 OM -->|Open Host Service
Published Language| Shipping[Shipping BC]
 Stripe((Stripe
External)) -.->|ACL| Payments
 ...
```

## Relationships

### Storefront → Order Management
- Pattern: Customer-Supplier
- Direction: Storefront depends on OM API contract
- Translation: none (shared types via Published Language)
- Spec file: /bounded-contexts/storefront/api/order-management-api.yaml

### Catalog → Order Management
- Pattern: Conformist + ACL (defensive)
- Reason: Catalog evolves independently; we accept its model but isolate it behind ACL to prevent leak of Catalog-specific terms (`SKU`, `MerchandiseCategory`) into our Order model.
- Spec file: /bounded-contexts/order-management/acl-catalog.md

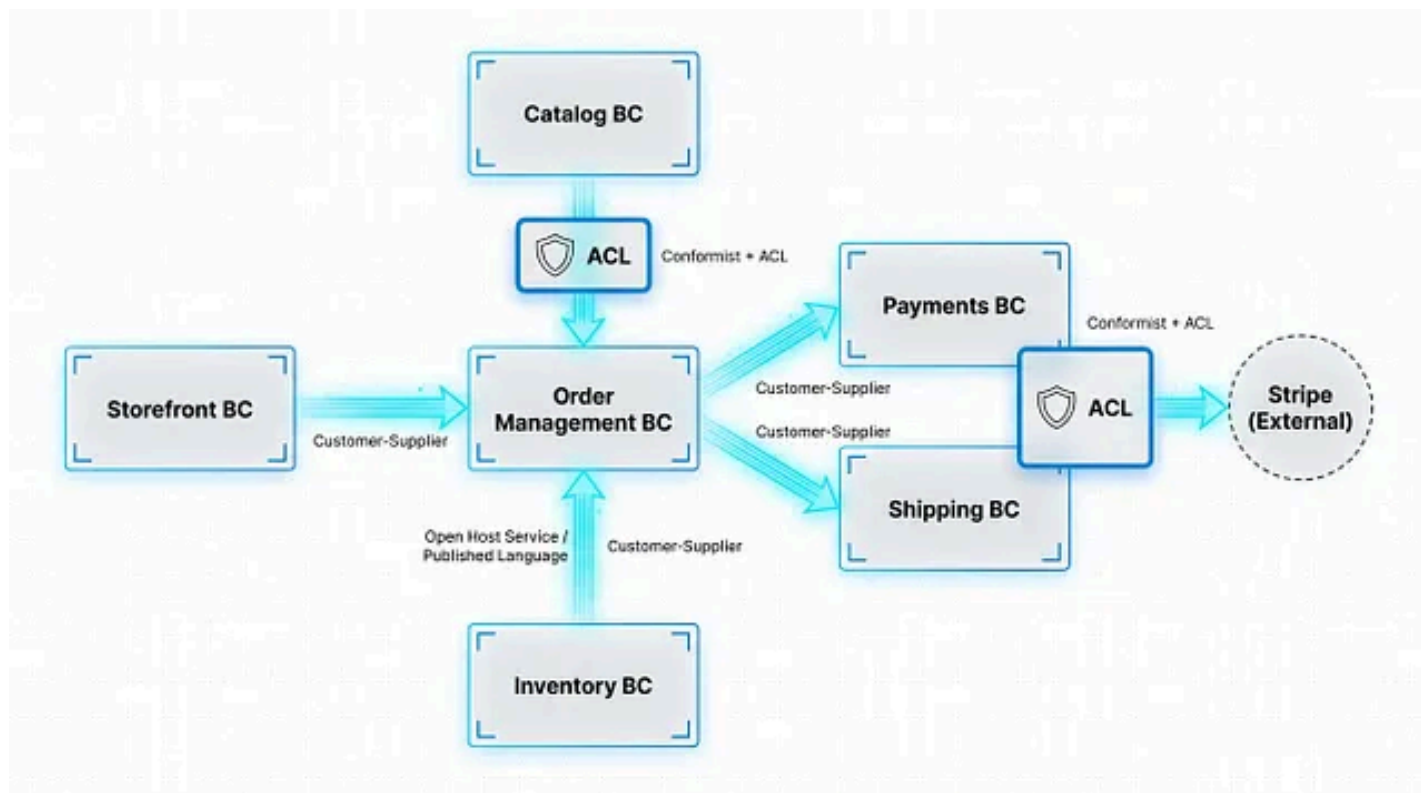
### Order Management → Payments
- Pattern: Open Host Service + Published Language
- Reason: Multiple downstream consumers (Payments, Shipping, Analytics) need stable, versioned event contracts.
- Spec file: /bounded-contexts/order-management/api/events.yaml

### Payments → Stripe (external)
- Pattern: ACL (mandatory – external)
- Reason: Stripe model leaks PaymentIntent/Charge/Refund concepts
```

that don't match our Payment aggregate. ACL translates.

- ****Spec file:**** /bounded-contexts/payments/acl-stripe.md

Each relationship carries one of seven patterns (Shared Kernel, Customer-Supplier, Conformist, ACL, Open Host Service, Published Language, Separate Ways), a justification for the choice, and a pointer to the specific spec file that enforces it.



The Context Map tells AI where to generate ACL (external integrations + defensive Conformist), where to use Open Host Service (multi-consumer events), and where to type through Published Language (shared schemas).

This is **the most important artifact for AI**. A structured aggregate spec with an "Enforced Invariants" section is what raises the quality of generated code from 60% to 80% of the human-written level. The format draws on the Codely Aggregate Design Blueprint and the Aggregate Design Canvas (DDD Crew).

```
# Aggregate: Order
```

```
## Description
```

```
Root aggregate of the Order Management BC. Captures customer's
purchase commitment with full price snapshot. Enforces all business
rules at the boundary.
```

```
## Aggregate Root
```

```
`Order` (entity)
```


Member Entities and Value Objects

- `OrderItem` (entity, owned)
- `Money` (value object – Amount + Currency)
- `Quantity` (value object – positive integer)
- `OrderId`, `CustomerId`, `ProductId` (value objects – identifiers)
- `OrderStatus` (enum: Draft | Confirmed | Fulfilled | Closed | Cancelled)

Properties

| Field | Type | Mutability |
|-------------|-----------------|---|
| Id | OrderId | immutable after creation |
| CustomerId | CustomerId | immutable after creation |
| Items | List<OrderItem> | mutable in Draft, immutable after Confirmed |
| Status | OrderStatus | mutable through commands only |
| TotalAmount | Money | computed (sum of items × prices) |
| ConfirmedAt | DateTime? | set on Confirm() |

Enforced Invariants

1. WHEN an Order receives `Confirm()` command IF Items.count == 0 THEN system SHALL reject with `EmptyOrderError`.
2. WHEN an Order receives `AddItem(productId, quantity)` IF Status != Draft THEN system SHALL reject with `OrderImmutableError`.
3. WHEN an Order receives `AddItem(productId, quantity)` IF quantity <= 0 THEN system SHALL reject with `InvalidQuantityError`.
4. THE Order's TotalAmount SHALL equal the sum of all OrderItems' (UnitPrice × Quantity), computed at read time.
5. WHEN an Order is Confirmed THE Items collection SHALL NOT be modifiable (no Add, no Remove, no quantity change).
6. THE Order SHALL NOT reference other Aggregates by direct object reference – only by identity (CustomerId, ProductId).

Corrective Policies (eventual consistency)

- IF a referenced Product is removed from Catalog AFTER Order Confirmation THE Order SHALL remain valid (price was snapshot at confirmation time).
- IF Customer is anonymized AFTER Order Confirmation THE Order SHALL retain CustomerId for audit, but emit `CustomerAnonymized` event for downstream PII handling.

Commands

| Command | Preconditions | Postconditions | Events emitted |
|---------------------------|----------------------------------|----------------------------|------------------|
| `CreateDraft(customerId)` | none | Status = Draft, Items = [] | OrderDrafted |
| `AddItem(productId, qty)` | Status == Draft, qty > 0 | item appended | OrderItemAdded |
| `RemoveItem(orderItemId)` | Status == Draft | item removed | OrderItemRemoved |
| `Confirm()` | Status == Draft, Items.count > 0 | Status = Confirmed | OrderConfirmed |

```

| `Cancel()` | Status in (Draft, Confirmed) | Status = Cancelled |
OrderCancelled |

## Domain Events Emitted
- `OrderDrafted(OrderId, CustomerId)`
- `OrderItemAdded(OrderId, OrderItemId, ProductId, Quantity,
UnitPrice)`
- `OrderItemRemoved(OrderId, OrderItemId)`
- `OrderConfirmed(OrderId, TotalAmount, ItemCount, ConfirmedAt)`
- `OrderCancelled(OrderId, CancelledAt, Reason)`

## Repository Interface
```csharp
interface IOrderRepository {
 Task<Order?> FindById(OrderId id);
 Task<List<Order>> FindByCustomer(CustomerId customerId);
 Task Save(Order order);
}
```

```

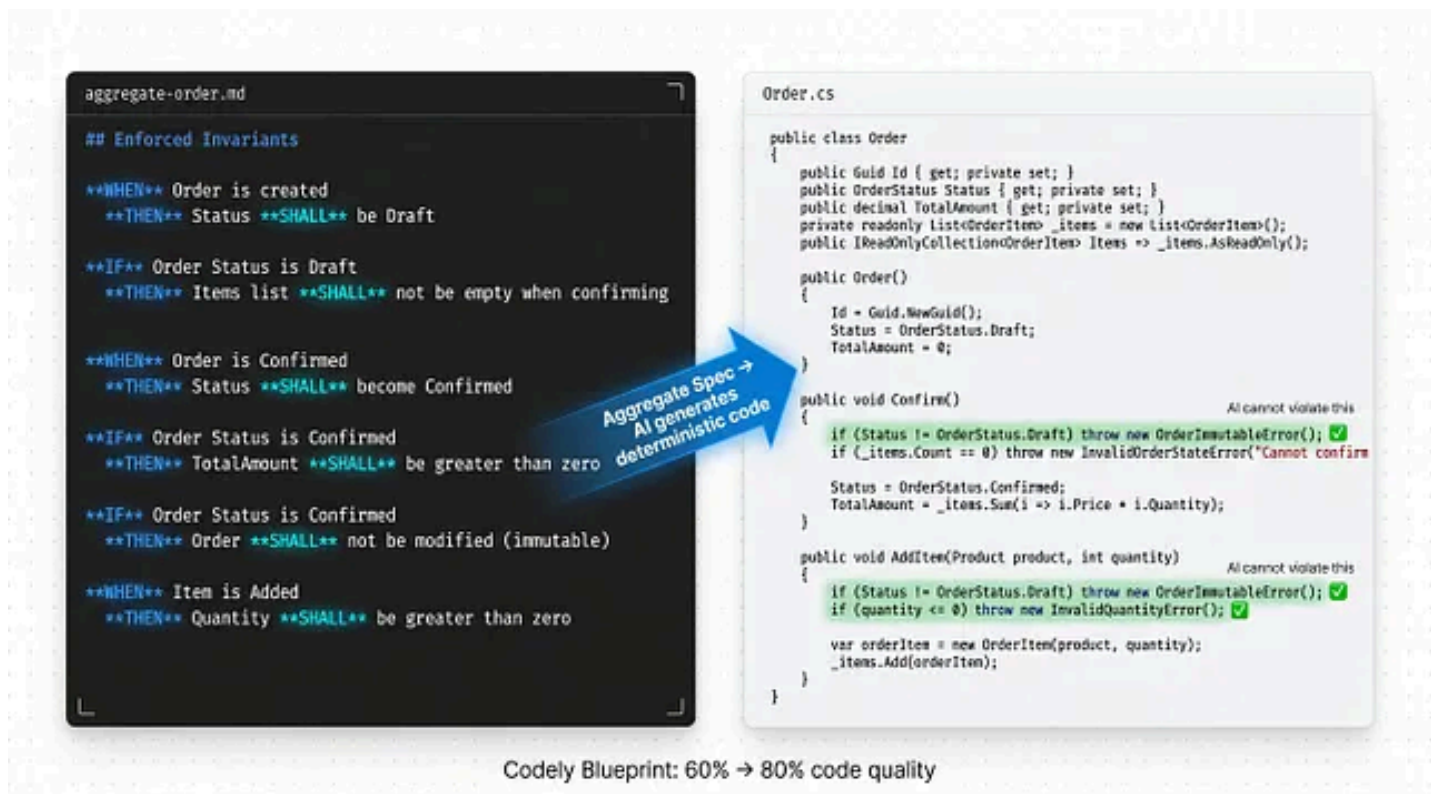
Three details that actually make a difference for AI generating code:

First — invariants in EARS notation (Easy Approach to Requirements Syntax — Alistair Mavin, Rolls-Royce, 2009). EARS defines five pattern types (Ubiquitous, Event-Driven, State-Driven, Unwanted Behavior, Optional Feature); for aggregate invariants I use a combination of Event-Driven + Conditional, with the keywords WHEN/IF/THEN/SHALL.

The notation is tool-independent but has been adopted natively by AWS Kiro and GitHub Spec Kit. **Every invariant is human-readable and structurally parseable by AI.** The classic `order.TotalAmount += price` bug without status validation disappears because invariant 2 is explicitly written.

Second — commands with preconditions and emitted events in a table. AI knows when it can call `Confirm()` (`Status == Draft, Items.count > 0`) and what it MUST emit on success. Without that table, the agent guesses or skips events entirely.

Third — the “Corrective Policies” section for eventual consistency. It tells AI what to do when invariant A of aggregate A collides with an event from aggregate B. Without it, the agent generates either multi-aggregate transactions (distributed monolith) or tight HTTP couplings between modules.



An Aggregate Spec with invariants in EARS is an executable contract — AI can't generate a method that violates an invariant because it's explicitly specified.

An operational decision that changed my SDD pipelines: **the Aggregate Design Spec is mandatory for every Core subdomain and is written by a human, not generated by AI.** Generic subdomains (auth, mailing) can stay anemic — no point in spending effort where there's no competitive advantage.

Artifact 5: Anti-Corruption Layer Spec → acl-{system}.md

The ACL Spec is mandatory for every external integration (Stripe, Salesforce, SAP) and everywhere the Context Map indicates a Conformist pattern with a defensive posture. **ACL protects your Aggregate from external model leakage into the core domain.**

Anti-Corruption Layer: Stripe Integration

Purpose

Translate Stripe payment concepts (PaymentIntent, Charge, Refund) into the Payments BC ubiquitous language (Payment, Refund, PaymentMethod). Prevent leak of Stripe-specific terms into core Payment aggregate.

Boundary

— ****Inside (our domain):**** `Payment` aggregate, `Money` VO, `PaymentId`,

```

`PaymentStatus` (Pending | Captured | Refunded | Failed)
- **Outside (Stripe):** `PaymentIntent`, `Charge`, `Refund`, `Customer`
  (Stripe), error codes (`card_declined`, `expired_card`, ...)

## Translation Map (inbound — Stripe → our domain)
Stripe field	Our field	Translation
`payment_intent.id`	`Payment.ExternalRef`	direct (string)
`payment_intent.amount`	`Payment.Amount`	Money.fromCents(cents, currency)
`payment_intent.currency`	`Payment.Amount.Currency`	uppercase ISO 4217
`payment_intent.status='succeeded'`	`Payment.Status=Captured`	enum mapping
`payment_intent.status='requires_action'`	`Payment.Status=Pending`	enum mapping
`payment_intent.status='canceled'`	`Payment.Status=Failed`	enum mapping
`charge.failure_code`	`Payment.FailureReason`	mapped via FailureMap below

## Translation Map (outbound — our domain → Stripe)
Our command	Stripe API call	Notes
`Payment.Capture(amount)`	POST /v1/payment_intents/{id}/capture	Idempotent
`Payment.Refund(amount, reason)`	POST /v1/refunds	reason maps to Stripe error code

## Error Translation
Stripe code	Our exception
`card_declined`	`PaymentDeclinedError`
`expired_card`	`PaymentDeclinedError` (subtype: ExpiredCard)
`insufficient_funds`	`PaymentDeclinedError` (subtype: InsufficientFunds)
`rate_limit`	`PaymentTransientError` (retry with backoff)
`api_error`	`PaymentTransientError` (retry with backoff)
anything else	`PaymentUnknownError` (alert + manual review)

## Idempotency
All write operations to Stripe MUST include `Idempotency-Key` header
set to `payment-{paymentId}-{commandName}-{commandVersion}`.

## Forbidden Concepts (do NOT leak into core domain)
- `PaymentIntent`, `Charge`, `Refund` (Stripe types)
- Stripe-specific status codes (`requires_action`, `requires_capture`)
- Stripe Customer object (we use our own CustomerId via metadata)

```

Three ACL Spec elements AI actually needs: **Translation Map** (field-to-field mapping), **Error Translation** (how to map external errors to our exceptions), and **Forbidden Concepts** (the list of external names AI must not use in the core domain). Without that last section, the agent generates `class PaymentIntent` in our domain, treating Stripe-specific concepts as our own model.

The /specs Folder for Spec First L1

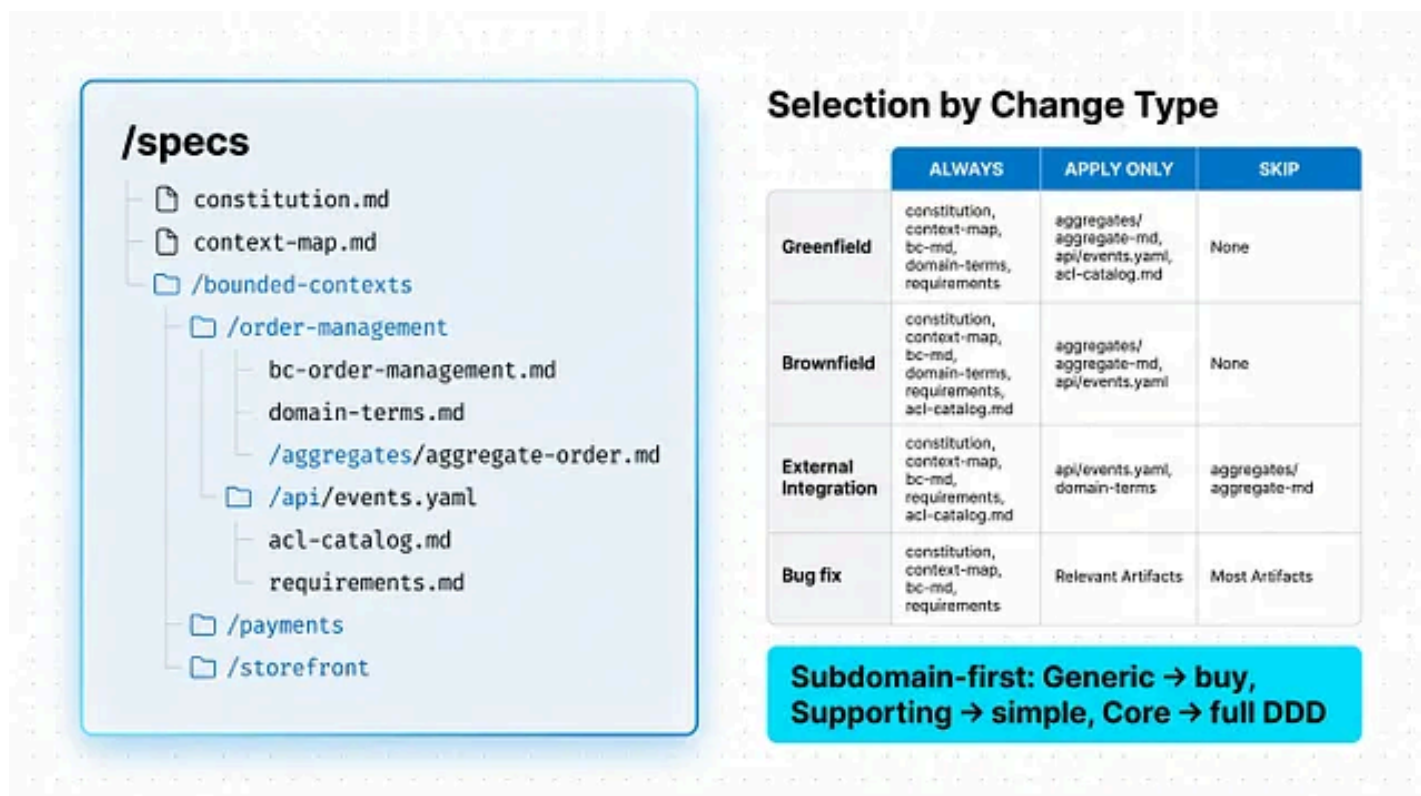
Everything assembles into one structure. The specific layout I use:

```

/specs
├── constitution.md           # inviolable principles
├── context-map.md           # BC relationships
├── /bounded-contexts
│   ├── /order-management
│   │   ├── bc-order-management.md    # BC Canvas
│   │   ├── domain-terms.md          # Ubiquitous Language
│   │   ├── /aggregates
│   │   │   ├── aggregate-order.md    # Aggregate Spec
│   │   │   └── aggregate-customer.md
│   │   ├── /api
│   │   │   ├── orders-api.yaml      # OpenAPI (HTTP)
│   │   │   └── events.yaml          # event schemas
│   │   ├── acl-catalog.md           # defensive ACL
│   │   └── requirements.md          # User Stories + EARS
│   ├── /payments
│   │   ├── bc-payments.md
│   │   ├── domain-terms.md
│   │   ├── /aggregates
│   │   │   └── aggregate-payment.md
│   │   ├── acl-stripe.md            # external ACL
│   │   └── requirements.md
│   └── /storefront
│       └── ...
└── /tasks
    └── ...                    # task decomposition (Spec Kit)

```

Artifact selection per change type in SDD L1 is pragmatic. **Greenfield (new feature)** — domain-terms + bc-canvas + aggregate-spec (if Core) + requirements. **Brownfield (refactor)** — bc-canvas (re-discovery of boundaries) + aggregate-roots (consistency boundaries) + acl if cutting out a legacy system. **External integration** — context-map update + acl-{external} + events schema. **Bug fix** — UL discipline only (right names in commits).



The `/specs` folder is an executable artifact — the agent loads it once and generates contextual code without re-explanation between sessions.

The `/specs` folder is versioned in Git, it's the source of truth for Claude Code agents (loaded at the start of every session as MCP context or via CLAUDE.md scope), and it's reviewed like code — a pull request changing the spec **MUST** precede a pull request changing the implementation. That's **Spec First**: the spec goes first, code consumes it.

The Boundary LLMs Won't Cross

One topic I won't skip, because it defines exactly who should be writing these templates. **LLMs are poor at *generating* tactical DDD; they're excellent at *consuming* tactical DDD as a spec.**

The FTAPI case study (arXiv 2603.26244, March 2026) testing a five-step LLM-driven DDD framework across three SOTA models showed that LLMs are highly effective at extracting UL from requirements documents, identifying domain events, and proposing BC boundaries (steps 1–3). They fail at aggregate design and mapping to technical architecture (steps 4–5) — through accumulated small errors. The authors note limited external validity (single case), but **the failure pattern in tactical steps is consistent with other industry observations.**

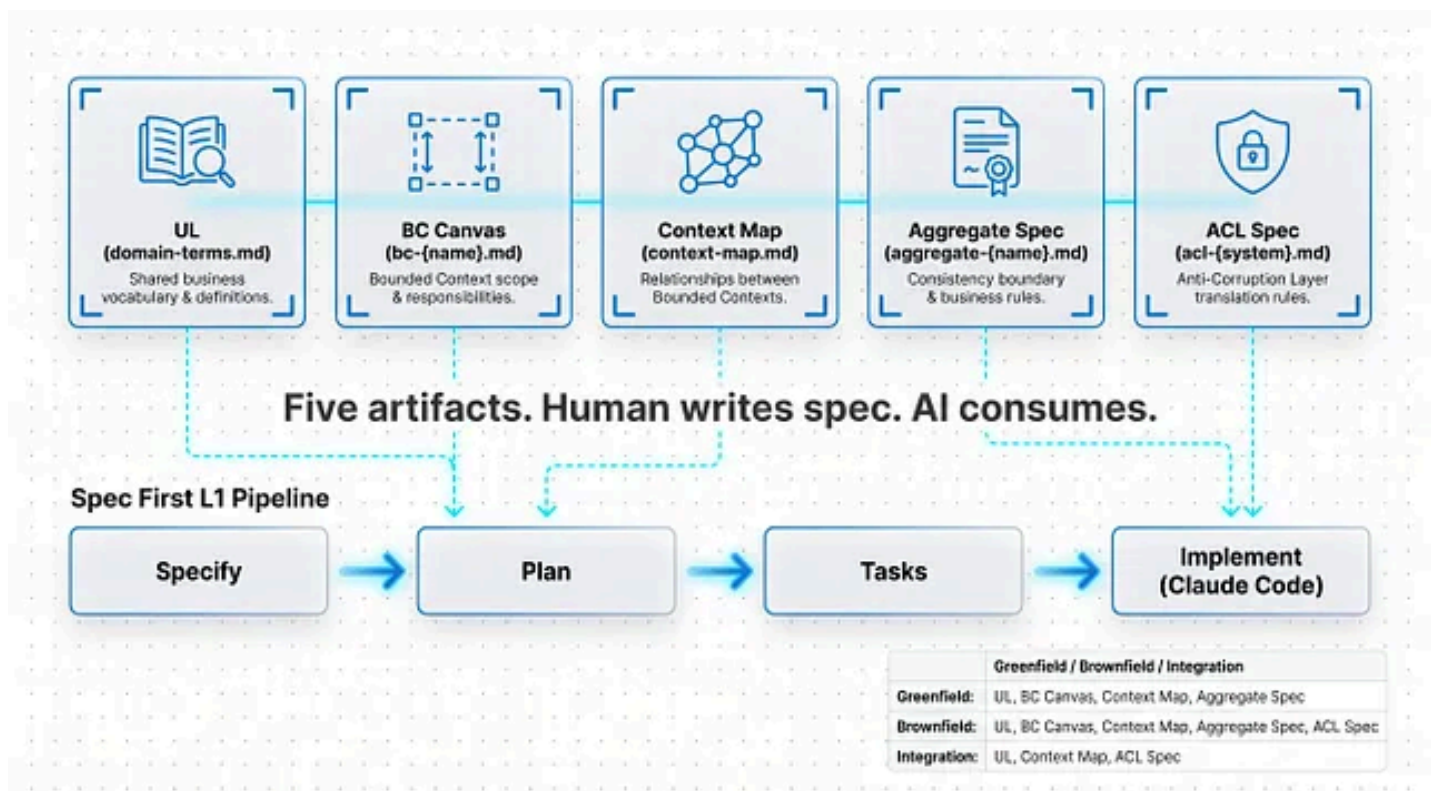
A practical consequence I've built into my SDD pipeline: **strategic artifacts (UL, BC Canvas, Context Map) can be drafted by a sub-agent, with a human verifying.** The “Aliases to AVOID” entries in UL, “Business Decisions” in BC Canvas, relationship patterns in the Context Map — reviewed and corrected manually. **Aggregate Design Spec and ACL Spec are written by a human from scratch.** That's where LLMs only consume.

The Aggregate Design Blueprint didn't generate aggregates — it was a **specification** structure that a human filled in. That's why it produces the 60% → 80% delta. Different direction of editing, different results.

Summary and Key Takeaways

Back to the PocketOS incident from the opening paragraph. Nine seconds in April 2026. Database and backups gone. Five files were missing from /specs — that's the operational thesis, not a historical footnote.

Without `domain-terms.md` the agent can't distinguish a production DELETE from a test one because terms are undefined. Without `aggregate-database-instance.md` with the invariant cannot delete production resource without confirmation token the agent has no contract it can't violate. **With those files, the invariant is explicitly written in EARS and AI has no way to bypass it.**



The article's mental model in one image: five DDD artifacts as Spec First specification elements, plus artifact selection per change type.

Four practical takeaways:

- **The five DDD artifacts as Spec First spec elements are UL, BC Canvas, Context Map, Aggregate Spec, and ACL.** Plus classic `constitution.md` and `requirements.md` in EARS. Each artifact eliminates a specific class of LLM hallucination.
- **Aggregate Design Spec with invariants in EARS notation is the strongest contract for AI.** Written by a human, consumed by an LLM — the empirical quality delta metric is in the aggregates section.
- **Artifact selection depends on the change type in SDD L1.** Greenfield needs UL + BC Canvas + Aggregate Spec + requirements. External integration needs Context Map update + ACL. Bug fix — UL discipline only.
- **DDD is not decoration for SDD Level 1 — it's its semantic layer.** Without it, `requirements.md` with EARS is ambiguous because terms aren't defined. EARS and DDD are complementary.

Four operational decisions:

- In your SDD repo, create the folder `/specs/{constitution.md, context-map.md, /bounded-contexts/{name}/{bc-canvas, domain-terms, aggregates, api, acl, requirements}.md}`
- In every new project, the first file is `domain-terms.md` per BC (not a database schema, not an API contract)
- Aggregate Spec with “Enforced Invariants” in EARS notation is mandatory for every Core subdomain — written by a human
- A pull request changing the spec **MUST** precede a pull request changing the implementation (the essence of Spec First L1)

“The whole is something beyond its parts.” — *paraphrase of Aristotle's Metaphysics, 1045a; closer original: “the whole is something beyond the parts”*

Writing five markdown templates before the first prompt doesn't slow you down in the AI era. **It's the only thing that actually speeds you up now.** Six months

working with agents in production taught me that without the DDD semantic layer, every hour saved on writing the spec comes back as five hours of debugging.